Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп’ютерних технологій, автоматики та метрології
Кафедра «Захист інформації»



Звіт

до лабораторної роботи №3

з курсу «Програмування скриптовими мовами»

на тему «Прикладні бібліотеки Python для побудови веб-застосунків»

Варіант 13

Виконав: студент групи КБ-202
Лозинський Р.Р.

Прийняв: Петрів П. П.

2026

1 Мета роботи

Навчитися створювати Web API за допомогою FastAPI, визначати маршрути та користуватися автоматичною документацією. Опанувати створення SQLite-бази й виконання параметризованих SQL-запитів. Реалізувати валідацію даних, моделі відповідей, обробку помилок і пов’язані зміни в межах однієї транзакції.

2 Предметна область і варіант 13

Варіант 13 — «Реєстр ризиків хмарної конфігурації». Сервіс зберігає навчальні знахідки аудиту. Категорія визначає напрям перевірки, value — оцінку ризику, status — стан опрацювання знахідки.

Поле або параметр	Значення
categories.name	iam, storage, network
records.code	Унікальний код V13-NN
records.title	Знахідка хмарного аудиту NN
records.category_id	Посилання на напрям аудиту
records.value	Скінченне float від 0 до 10 включно
records.status	new, in_progress, done
Одиниця unit	score
Поріг	7.0; вибірка GET /records?min_value=7

Ідентифікатори та offset обмежено максимальним SQLite INTEGER 9223372036854775807. Перевищення повертає 422. Поріг використовується для відбору знахідок з оцінкою не меншою за 7.0. Він не обмежує допустимі дані: оцінки нижче порога також зберігаються. Оцінки є навчальними й не розраховуються за окремою стандартизованою методикою.

3 Підготовка проєкту

Підготовлено пакет labs/lab03 для репозиторію cybersecurity-python-labs-2. Матеріали labs/lab01 збережено. Дані студента імпортуються з наявного shared/student.py як пакет без ручного редагування sys.path. Використано Python 3.10.7 та Pydantic 2.

Репозиторій: https://github.com/renduishka/cybersecurity-python-labs-2

```
fastapi==0.128.0
uvicorn==0.40.0
pydantic==2.10.6
ruff==0.17.1
httpx==0.28.1
```

Для Ruff збережено правила E, W, F, I, N і довжину рядка 79 символів із попередньої роботи. Робочі SQLite-файли, віртуальне середовище та кеші виключено через .gitignore. Навчальний seed.json зберігається разом із кодом.

```
cybersecurity-python-labs-2/
├── .gitignore
├── requirements.txt
├── ruff.toml
├── shared/
│   ├── __init__.py
```

```
│   └── student.py
├── labs/
│   ├── __init__.py
│   ├── lab01/                  # збережена попередня робота
│   └── lab03/
│       ├── __init__.py
│       ├── main.py
│       ├── database.py
│       ├── schema.sql
│       ├── repository.py
│       ├── schemas.py
│       ├── config.py
│       ├── seed.py
│       ├── routers/
│       │   ├── __init__.py
│       │   ├── categories.py
│       │   └── records.py
│       ├── data/seed.json
│       └── README.md
└── tests/
    ├── test_lab03.py
    └── verify_http.py
```

Команди запуску

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m labs.lab03.database
python -m labs.lab03.seed
uvicorn labs.lab03.main:app --reload --host 127.0.0.1 --port 8000
```

Swagger UI: http://127.0.0.1:8000/docs. OpenAPI: http://127.0.0.1:8000/openapi.json. Шлях до data/lab3.sqlite3 визначається від database.py, тому не залежить від поточної робочої директорії.

4 Схема бази даних

Використано без зміни SQL-команд надану схему: спільний початок, блок records для варіантів 13–15 з оцінками 0–10 і спільне завершення. Зв’язок categories → records — «один до багатьох».

Таблиця	Ключі та обмеження
categories	PK id > 0; name NOT NULL UNIQUE; довжина trim(name) 2–40
records	PK id > 0; code NOT NULL UNIQUE (3–40); title NOT NULL (3–120); category_id INTEGER > 0; value REAL 0–10; status IN (new, in_progress, done)
Зв’язок	records.category_id → categories.id; FOREIGN KEY, ON DELETE RESTRICT

```
PRAGMA foreign_keys = ON;
BEGIN TRANSACTION;
CREATE TABLE IF NOT EXISTS categories (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    name TEXT NOT NULL UNIQUE
        CHECK (length(trim(name)) BETWEEN 2 AND 40)
);
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    code TEXT NOT NULL UNIQUE
        CHECK (length(trim(code)) BETWEEN 3 AND 40),
    title TEXT NOT NULL
        CHECK (length(trim(title)) BETWEEN 3 AND 120),
    category_id INTEGER NOT NULL
```

```
        CHECK (typeof(category_id) = 'integer' AND category_id > 0),
    value REAL NOT NULL
        CHECK (typeof(value) IN ('integer', 'real')
               AND value BETWEEN 0 AND 10),
    status TEXT NOT NULL
        CHECK (status IN ('new', 'in_progress', 'done')),
    FOREIGN KEY (category_id) REFERENCES categories(id)
        ON DELETE RESTRICT
);
COMMIT;
```

ON DELETE RESTRICT захищає непорожню категорію від видалення. PRAGMA foreign_keys = ON виконується для кожного нового з’єднання до початку транзакції. CHECK і UNIQUE доповнюють валідацію Pydantic.

5 Лістинги основних модулів

Дані студента

shared/student.py

```
STUDENT_NAME = "Лозинський Ростислав Романович"
GROUP_NAME = "КБ-202"
VARIANT_NUMBER = 13
```

Параметри варіанта

labs/lab03/config.py

```
"""Варіант 13: реєстр ризиків хмарної конфігурації."""

VARIANT_TITLE = "Реєстр ризиків хмарної конфігурації"
DOMAIN_DESCRIPTION = (
    "Облік навчальних знахідок аудиту хмарної конфігурації. "
    "Категорії: iam, storage, network. value — оцінка ризику від 0 до 10; "
    "status — стан опрацювання. Оцінки не є розрахунком за стандартом CVSS."
)
UNIT = "score"
THRESHOLD = 7.0
VALUE_MIN = 0.0
VALUE_MAX = 10.0
DEFAULT_LIMIT = 20
MAX_LIMIT = 100
BULK_MIN = 1
BULK_MAX = 100
CATEGORY_NAME_MIN = 2
CATEGORY_NAME_MAX = 40
CODE_MIN = 3
CODE_MAX = 40
TITLE_MIN = 3
TITLE_MAX = 120

# SQLite INTEGER: знакове 64-бітне ціле.
SQLITE_INT_MAX = 9223372036854775807
```

Створення бази та з’єднання

labs/lab03/database.py

```
"""З'єднання SQLite та виконання наданого SQL-скрипту."""

import sqlite3
from contextlib import closing
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent
DATABASE_PATH = BASE_DIR / "data" / "lab3.sqlite3"
SCHEMA_PATH = BASE_DIR / "schema.sql"


def get_connection() -> sqlite3.Connection:
    """Відкрити з'єднання з перевіркою зовнішніх ключів."""
    DATABASE_PATH.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(DATABASE_PATH)
    try:
        connection.execute("PRAGMA foreign_keys = ON")
        connection.row_factory = sqlite3.Row
        return connection
    except sqlite3.Error:
        connection.close()
        raise


def initialize_database() -> None:
    """Створити таблиці, не змінюючи наявних даних."""
    script = SCHEMA_PATH.read_text(encoding="utf-8")
    with closing(get_connection()) as connection:
        try:
            connection.executescript(script)
        except sqlite3.Error:
            connection.rollback()
            raise


if __name__ == "__main__":
    initialize_database()
    print("Базу SQLite створено; наявні дані збережено.")
```

Моделі запитів і відповідей

labs/lab03/schemas.py

```
"""Моделі даних API та перевірки початкового набору."""

from typing import Literal

from pydantic import BaseModel, ConfigDict, Field, model_validator

from labs.lab03 import config

Status = Literal["new", "in_progress", "done"]


class StrictInput(BaseModel):
    """Вхідні дані без зайвих полів та крайових пробілів."""

    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")


class CategoryCreate(StrictInput):
    """Назва напряму аудиту."""

    name: str = Field(
        min_length=config.CATEGORY_NAME_MIN,
        max_length=config.CATEGORY_NAME_MAX,
    )


class CategoryRead(CategoryCreate):
    """Категорія з ідентифікатором SQLite."""

    id: int = Field(gt=0, le=config.SQLITE_INT_MAX)


class RecordCreate(StrictInput):
    """Новий результат аудиту; id визначає база."""
```

```
    code: str = Field(
        min_length=config.CODE_MIN,
        max_length=config.CODE_MAX,
        examples=["V13-TEST"],
    )
    title: str = Field(
        min_length=config.TITLE_MIN, max_length=config.TITLE_MAX
    )
    category_id: int = Field(
        gt=0,
        le=config.SQLITE_INT_MAX,
        description="Ідентифікатор напряму аудиту",
    )
    value: float = Field(
        ge=config.VALUE_MIN,
        le=config.VALUE_MAX,
        allow_inf_nan=False,
        description="Навчальна оцінка ризику, score (0–10)",
    )
    status: Status


class RecordReplace(RecordCreate):
    """Повна заміна всіх п'яти змінюваних полів запису."""


class RecordRead(RecordCreate):
    """Результат JOIN із назвою категорії."""

    id: int = Field(gt=0, le=config.SQLITE_INT_MAX)
    category_name: str


class StudentRead(BaseModel):
    """Відомості про виконавця."""

    student_name: str
    group_name: str
    variant_number: int


class PingRead(BaseModel):
    """Ознака доступності API."""

    status: Literal["ok"]


class SeedData(StrictInput):
    """Три категорії та вісім навчальних знахідок."""

    categories: list[CategoryRead] = Field(min_length=3, max_length=3)
    records: list[RecordCreate] = Field(min_length=8, max_length=8)

    @model_validator(mode="after")
    def check_relations(self) -> "SeedData":
        """Відхилити дублікати та відсутні посилання до запису в БД."""
        ids = [category.id for category in self.categories]
        names = [category.name for category in self.categories]
        codes = [record.code for record in self.records]
        if len(set(ids)) != 3 or len(set(names)) != 3 or len(set(codes)) != 8:
            raise ValueError("Початковий набір містить дублікати")
        if any(record.category_id not in ids for record in self.records):
            raise ValueError("Категорію початкового запису не знайдено")
        return self
```

Параметризовані SQL та транзакції

labs/lab03/repository.py

```
"""Параметризовані SQL-запити; цей модуль не залежить від FastAPI."""

import sqlite3
```

```
from contextlib import closing
from typing import Any

from labs.lab03.database import get_connection

SELECT_RECORD = (
    "SELECT r.id, r.code, r.title, r.category_id, r.value, r.status, "
    "c.name AS category_name FROM records AS r "
    "JOIN categories AS c ON c.id = r.category_id"
)
INSERT_RECORD = (
    "INSERT INTO records (code, title, category_id, value, status) "
    "VALUES (?, ?, ?, ?, ?)"
)


class NotFoundError(Exception):
    """Ресурс або потрібна категорія не існує."""


def _require_category(
    connection: sqlite3.Connection, category_id: int
) -> None:
    if (
        connection.execute(
            "SELECT id FROM categories WHERE id = ?", (category_id,)
        ).fetchone()
        is None
    ):
        raise NotFoundError("Категорію не знайдено")


def _read_record(
    connection: sqlite3.Connection, record_id: int
) -> dict[str, Any]:
    row = connection.execute(
        SELECT_RECORD + " WHERE r.id = ?", (record_id,)
    ).fetchone()
    if row is None:
        raise NotFoundError("Запис не знайдено")
    return dict(row)


def _values(record: dict[str, Any]) -> tuple[Any, ...]:
    return tuple(
        record[key]
        for key in ("code", "title", "category_id", "value", "status")
    )


def list_categories() -> list[dict[str, Any]]:
    """Отримати категорії у стабільному порядку."""
    with closing(get_connection()) as connection:
        return [
            dict(row)
            for row in connection.execute(
                "SELECT id, name FROM categories ORDER BY id"
            )
        ]


def create_category(name: str) -> dict[str, Any]:
    """Додати категорію та підтвердити транзакцію."""
    with closing(get_connection()) as connection, connection:
        cursor = connection.execute(
            "INSERT INTO categories (name) VALUES (?)", (name,)
        )
        return dict(
            connection.execute(
                "SELECT id, name FROM categories WHERE id = ?",
                (cursor.lastrowid,),
            ).fetchone()
        )
```

```
def delete_category(category_id: int) -> None:
    """Видалити порожню категорію; зовнішній ключ захищає непорожню."""
    with closing(get_connection()) as connection, connection:
        cursor = connection.execute(
            "DELETE FROM categories WHERE id = ?", (category_id,)
        )
        if cursor.rowcount == 0:
            raise NotFoundError("Категорію не знайдено")


def get_record(record_id: int) -> dict[str, Any]:
    """Прочитати одну знахідку з назвою напряму аудиту."""
    with closing(get_connection()) as connection:
        return _read_record(connection, record_id)


def list_records(
    category_id: int | None = None,
    status: str | None = None,
    min_value: float | None = None,
    q: str | None = None,
    limit: int = 20,
    offset: int = 0,
) -> list[dict[str, Any]]:
    """Застосувати AND-фільтри, буквальний пошук та пагінацію."""
    clauses: list[str] = []
    params: list[Any] = []
    for clause, value in [
        ("r.category_id = ?", category_id),
        ("r.status = ?", status),
        ("r.value >= ?", min_value),
        ("instr(r.title, ?) > 0", q),
    ]:
        if value is not None:
            clauses.append(clause)
            params.append(value)
    # Структура SQL складається лише з фіксованих фрагментів коду.
    query = SELECT_RECORD
    if clauses:
        query += " WHERE " + " AND ".join(clauses)
    query += " ORDER BY r.id LIMIT ? OFFSET ?"
    params.extend((limit, offset))
    with closing(get_connection()) as connection:
        return [dict(row) for row in connection.execute(query, params)]


def create_records(records: list[dict[str, Any]]) -> list[dict[str, Any]]:
    """Вставити весь пакет в одній транзакції або скасувати всі вставки."""
    with closing(get_connection()) as connection, connection:
        connection.execute("BEGIN")
        result = []
        for record in records:
            _require_category(connection, record["category_id"])
            cursor = connection.execute(INSERT_RECORD, _values(record))
            result.append(_read_record(connection, cursor.lastrowid))
        return result


def create_record(record: dict[str, Any]) -> dict[str, Any]:
    """Створити одну знахідку через спільну транзакційну операцію."""
    return create_records([record])[0]


def replace_record(record_id: int, record: dict[str, Any]) -> dict[str, Any]:
    """Замінити змінювані поля, зберігши id."""
    with closing(get_connection()) as connection, connection:
        connection.execute("BEGIN")
        _read_record(connection, record_id)
        _require_category(connection, record["category_id"])
        connection.execute(
            "UPDATE records SET code = ?, title = ?, category_id = ?, "
            "value = ?, status = ? WHERE id = ?",
            (*_values(record), record_id),
        )
        return _read_record(connection, record_id)
```

```
def delete_record(record_id: int) -> None:
    """Видалити знахідку або повідомити про відсутній id."""
    with closing(get_connection()) as connection, connection:
        cursor = connection.execute(
            "DELETE FROM records WHERE id = ?", (record_id,)
        )
        if cursor.rowcount == 0:
            raise NotFoundError("Запис не знайдено")
```

Початкове заповнення

labs/lab03/seed.py

```
"""Валідація JSON та атомарне початкове заповнення SQLite."""

import logging
import sqlite3
from contextlib import closing
from pathlib import Path

from labs.lab03.database import BASE_DIR, get_connection, initialize_database
from labs.lab03.schemas import SeedData

SEED_PATH = BASE_DIR / "data" / "seed.json"


def seed_database(path: Path = SEED_PATH) -> bool:
    """Заповнити лише порожню базу; повернути False, якщо вона непорожня."""
    initialize_database()
    with closing(get_connection()) as connection, connection:
        # Блокування запису усуває гонку між перевіркою порожнечі та вставками.
        connection.execute("BEGIN IMMEDIATE")
        occupied = connection.execute(
            "SELECT EXISTS(SELECT 1 FROM categories) "
            "OR EXISTS(SELECT 1 FROM records)"
        ).fetchone()[0]
        if occupied:
            return False
        data = SeedData.model_validate_json(path.read_text(encoding="utf-8"))
        connection.executemany(
            "INSERT INTO categories (id, name) VALUES (?, ?)",
            [(category.id, category.name) for category in data.categories],
        )
        connection.executemany(
            "INSERT INTO records (code, title, category_id, value, status) "
            "VALUES (?, ?, ?, ?, ?)",
            [
                (r.code, r.title, r.category_id, r.value, r.status)
                for r in data.records
            ],
        )
    return True


def main() -> int:
    """Надрукувати зрозумілий результат початкового заповнення."""
    try:
        inserted = seed_database()
    except (ValueError, OSError, sqlite3.Error):
        logging.exception("Помилка початкового заповнення")
        print("Початковий набір не додано. Перевірте JSON та доступ до бази.")
        return 1
    print(
        "Додано 3 категорії та 8 записів."
        if inserted
        else "База непорожня. Дані не змінено."
    )
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Маршрути категорій

labs/lab03/routers/categories.py

```
"""HTTP-маршрути категорій аудиту."""

from typing import Annotated

from fastapi import APIRouter, Path, Response

from labs.lab03 import config, repository
from labs.lab03.schemas import CategoryCreate, CategoryRead

router = APIRouter(prefix="/categories", tags=["Категорії"])


@router.get(
    "", response_model=list[CategoryRead], summary="Список напрямів аудиту"
)
def list_categories() -> list[dict]:
    """Повернути категорії за зростанням id."""
    return repository.list_categories()


@router.post(
    "",
    response_model=CategoryRead,
    status_code=201,
    summary="Створити напрям аудиту",
    responses={409: {"description": "Назва вже існує"}},
)
def create_category(data: CategoryCreate) -> dict:
    """Створити категорію з унікальною назвою."""
    return repository.create_category(data.name)


@router.delete(
    "/{category_id}",
    status_code=204,
    summary="Видалити порожню категорію",
    responses={
        404: {"description": "Категорію не знайдено"},
        409: {"description": "Категорія містить записи"},
    },
)
def delete_category(
    category_id: Annotated[
        int,
        Path(
            gt=0, le=config.SQLITE_INT_MAX, description="Додатний id категорії"
        ),
    ],
) -> Response:
    """Не дозволяти видалення категорії, яка має записи."""
    repository.delete_category(category_id)
    return Response(status_code=204)
```

Маршрути знахідок

labs/lab03/routers/records.py

```
"""HTTP-маршрути знахідок аудиту хмарної конфігурації."""

from typing import Annotated

from fastapi import APIRouter, Body, Path, Query, Response
```

```
from labs.lab03 import config, repository
from labs.lab03.schemas import RecordCreate, RecordRead, RecordReplace, Status

router = APIRouter(prefix="/records", tags=["Знахідки"])
NOT_FOUND = {404: {"description": "Запис або категорію не знайдено"}}
WRITE_ERRORS = {**NOT_FOUND, 409: {"description": "Унікальний code вже існує"}}
RecordId = Annotated[
    int,
    Path(gt=0, le=config.SQLITE_INT_MAX, description="Додатний id знахідки"),
]


@router.get(
    "", response_model=list[RecordRead], summary="Знайти знахідки з фільтрами"
)
def list_records(
    category_id: Annotated[
        int | None,
        Query(gt=0, le=config.SQLITE_INT_MAX, description="Напрям аудиту"),
    ] = None,
    status: Annotated[
        Status | None, Query(description="Стан опрацювання")
    ] = None,
    min_value: Annotated[
        float | None,
        Query(
            ge=config.VALUE_MIN,
            le=config.VALUE_MAX,
            allow_inf_nan=False,
            description="Мінімальна оцінка ризику (score)",
        ),
    ] = None,
    q: Annotated[
        str | None,
        Query(
            min_length=1,
            max_length=config.TITLE_MAX,
            description="Буквальний підрядок title з урахуванням регістру",
        ),
    ] = None,
    limit: Annotated[
        int, Query(ge=1, le=config.MAX_LIMIT, description="Розмір сторінки")
    ] = config.DEFAULT_LIMIT,
    offset: Annotated[
        int,
        Query(
            ge=0,
            le=config.SQLITE_INT_MAX,
            description="Кількість пропущених записів",
        ),
    ] = 0,
) -> list[dict]:
    """Поєднати задані умови через AND, сортувати за id."""
    return repository.list_records(
        category_id, status, min_value, q, limit, offset
    )


@router.post(
    "/bulk",
    response_model=list[RecordRead],
    status_code=201,
    responses=WRITE_ERRORS,
    summary="Створити пакет атомарно",
)
def create_bulk(
    records: Annotated[
        list[RecordCreate],
        Body(min_length=config.BULK_MIN, max_length=config.BULK_MAX),
    ],
) -> list[dict]:
    """Валідувати весь масив; за будь-якої помилки скасувати весь пакет."""
    return repository.create_records(
        [record.model_dump() for record in records]
    )
```

```
@router.post(
    "",
    response_model=RecordRead,
    status_code=201,
    responses=WRITE_ERRORS,
    summary="Створити знахідку",
)
def create_record(record: RecordCreate) -> dict:
    """Додати знахідку для наявної категорії."""
    return repository.create_record(record.model_dump())


@router.get(
    "/{record_id}",
    response_model=RecordRead,
    responses=NOT_FOUND,
    summary="Отримати знахідку за id",
)
def get_record(record_id: RecordId) -> dict:
    """Повернути знахідку з назвою категорії."""
    return repository.get_record(record_id)


@router.put(
    "/{record_id}",
    response_model=RecordRead,
    responses=WRITE_ERRORS,
    summary="Повністю замінити поля знахідки",
)
def replace_record(record_id: RecordId, record: RecordReplace) -> dict:
    """Оновити п'ять полів; id залишається незмінним."""
    return repository.replace_record(record_id, record.model_dump())


@router.delete(
    "/{record_id}",
    status_code=204,
    responses=NOT_FOUND,
    summary="Видалити знахідку",
)
def delete_record(record_id: RecordId) -> Response:
    """Видалити запис і повернути порожню відповідь."""
    repository.delete_record(record_id)
    return Response(status_code=204)
```

Застосунок та обробка помилок

labs/lab03/main.py

```
"""FastAPI-застосунок для лабораторної роботи 3, варіант 13."""

import logging
import math
import sqlite3
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from typing import Any

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

from labs.lab03 import config, database, repository
from labs.lab03.routers import categories, records
from labs.lab03.schemas import PingRead, StudentRead
from shared.student import GROUP_NAME, STUDENT_NAME, VARIANT_NUMBER

logger = logging.getLogger(__name__)
```

```
@asynccontextmanager
async def lifespan(application: FastAPI) -> AsyncIterator[None]:
    """Створити відсутні таблиці один раз під час запуску."""
    database.initialize_database()
    yield


app = FastAPI(
    title=config.VARIANT_TITLE,
    description=(
        f"{config.DOMAIN_DESCRIPTION} Одиниця: {config.UNIT}. "
        f"Поріг: {config.THRESHOLD}; вибірка: /records?min_value=7."
    ),
    version="1.0.0",
    lifespan=lifespan,
)
app.include_router(categories.router)
app.include_router(records.router)


@app.exception_handler(repository.NotFoundError)
def not_found(
    request: Request, error: repository.NotFoundError
) -> JSONResponse:
    """Перетворити відсутність ресурсу на HTTP 404."""
    return JSONResponse(status_code=404, content={"detail": str(error)})


@app.exception_handler(sqlite3.IntegrityError)
def integrity_error(
    request: Request, error: sqlite3.IntegrityError
) -> JSONResponse:
    """Повернути конфлікт без розкриття SQL і структури файлів."""
    logger.warning("Порушення цілісності SQLite: %s", error)
    return JSONResponse(
        status_code=409,
        content={
            "detail": "Конфлікт даних: дублікат або категорія містить записи"
        },
    )


@app.exception_handler(sqlite3.Error)
def database_error(request: Request, error: sqlite3.Error) -> JSONResponse:
    """Зберегти технічні подробиці лише в журналі сервера."""
    logger.error(
        "Помилка SQLite", exc_info=(type(error), error, error.__traceback__)
    )
    return JSONResponse(
        status_code=500, content={"detail": "Помилка роботи з базою даних"}
    )


def _json_safe(value: Any) -> Any:
    """Зробити повідомлення валідації серіалізовним навіть для NaN/inf."""
    if isinstance(value, float) and not math.isfinite(value):
        return str(value)
    if isinstance(value, dict):
        return {key: _json_safe(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [_json_safe(item) for item in value]
    if isinstance(value, Exception):
        return str(value)
    return value


@app.exception_handler(RequestValidationError)
def validation_error(
    request: Request, error: RequestValidationError
) -> JSONResponse:
    """Відхилити некоректні дані з JSON-відповіддю 422."""
    return JSONResponse(
        status_code=422, content={"detail": _json_safe(error.errors())}
    )
```

```
@app.get(
    "/ping",
    response_model=PingRead,
    tags=["Службові"],
    summary="Перевірити доступність",
)
def ping() -> dict[str, str]:
    """Повернути ознаку доступності API."""
    return {"status": "ok"}


@app.get(
    "/student",
    response_model=StudentRead,
    tags=["Службові"],
    summary="Дані студента",
)
def student() -> dict[str, Any]:
    """Прочитати дані виконавця зі спільного пакета."""
    return {
        "student_name": STUDENT_NAME,
        "group_name": GROUP_NAME,
        "variant_number": VARIANT_NUMBER,
    }
```

6 Початкові дані варіанта 13

id категорії	Назва
1	iam
2	storage
3	network

Назви записів: «Знахідка хмарного аудиту 01» … «Знахідка хмарного аудиту 08». Ідентифікатори записів створює SQLite; у JSON вони відсутні.

code	category_id	value	status
V13-01	1	8.0	new
V13-02	2	6.5	in_progress
V13-03	3	4.0	done
V13-04	1	9.0	new
V13-05	2	7.5	done
V13-06	3	3.0	in_progress
V13-07	1	7.0	done
V13-08	2	5.0	new

Seed перевіряє кількість елементів, типи, діапазони, унікальність id, назв і code та наявність категорій. BEGIN IMMEDIATE охоплює перевірку порожнечі й обидві вставки, тому паралельне заповнення не створить частковий набір. Для непорожньої бази функція повертає False без змін.

```
$ python -m labs.lab03.seed
Додано 3 категорії та 8 записів.
$ python -m labs.lab03.seed
База непорожня. Дані не змінено.
```

7 Перевірка API та збереження даних

Перевірки виконано 06.11.2026. Автоматизований HTTP-сценарій запускає справжній uvicorn на 127.0.0.1:8013 з окремою тимчасовою SQLite-базою. Тести не змінюють робочу базу. Зафіксовано 37 кроків усіх 14 сценаріїв методички. Нижче наведено фактичні запити, коди відповідей та результати. Повні відповіді збережено в output/evidence/http-results.json і http-transcript.txt.

Сценарій 1 Початкове заповнення та повторний запуск


```
{
  "first_inserted": true,
  "second_inserted": false,
  "categories": 3,
  "records": 8
}
```


Початкові 8 знахідок


```
GET /records?limit=100
HTTP 200
[
  {"code": "V13-01", "title": "Знахідка хмарного аудиту 01", "category_id": 1, "value":
8.0, "status": "new", "id": 1, "category_name": "iam"},
  {"code": "V13-02", "title": "Знахідка хмарного аудиту 02", "category_id": 2, "value":
6.5, "status": "in_progress", "id": 2, "category_name": "storage"},
  {"code": "V13-03", "title": "Знахідка хмарного аудиту 03", "category_id": 3, "value":
4.0, "status": "done", "id": 3, "category_name": "network"},
  {"code": "V13-04", "title": "Знахідка хмарного аудиту 04", "category_id": 1, "value":
9.0, "status": "new", "id": 4, "category_name": "iam"},
  {"code": "V13-05", "title": "Знахідка хмарного аудиту 05", "category_id": 2, "value":
7.5, "status": "done", "id": 5, "category_name": "storage"},
  {"code": "V13-06", "title": "Знахідка хмарного аудиту 06", "category_id": 3, "value":
3.0, "status": "in_progress", "id": 6, "category_name": "network"},
  {"code": "V13-07", "title": "Знахідка хмарного аудиту 07", "category_id": 1, "value":
7.0, "status": "done", "id": 7, "category_name": "iam"},
  {"code": "V13-08", "title": "Знахідка хмарного аудиту 08", "category_id": 2, "value":
5.0, "status": "new", "id": 8, "category_name": "storage"}
]
```


Сценарій 2 Службові маршрути та категорії

Перевірка доступності


```
GET /ping
HTTP 200
{"status": "ok"}
```


Дані студента


```
GET /student
HTTP 200
{"student_name": "Лозинський Ростислав Романович", "group_name": "КБ-202",
"variant_number": 13}
```


Напрями аудиту


```
GET /categories
HTTP 200
[{"name": "iam", "id": 1}, {"name": "storage", "id": 2}, {"name": "network", "id": 3}]
```

Сценарій 3 Повний цикл CRUD

Створення знахідки

```
POST /records
{"code": "V13-TEST", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new"}
HTTP 201
{"code": "V13-TEST", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new", "id": 9, "category_name": "iam"}
```

Читання знахідки

```
GET /records/9
HTTP 200
{"code": "V13-TEST", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new", "id": 9, "category_name": "iam"}
```

Повна заміна

```
PUT /records/9
{"code": "V13-TEST", "title": "Навчальна знахідка", "category_id": 1, "value": 2,
"status": "done"}
HTTP 200
{"code": "V13-TEST", "title": "Навчальна знахідка", "category_id": 1, "value": 2.0,
"status": "done", "id": 9, "category_name": "iam"}
```

Видалення без тіла

```
DELETE /records/9
HTTP 204
(порожнє тіло відповіді)
```

Сценарій 4 Фільтрація та пагінація

Фільтрація та пагінація

```
GET /records?category_id=1
HTTP 200
[
  {"code": "V13-01", "title": "Знахідка хмарного аудиту 01", "category_id": 1, "value":
8.0, "status": "new", "id": 1, "category_name": "iam"},
  {"code": "V13-04", "title": "Знахідка хмарного аудиту 04", "category_id": 1, "value":
9.0, "status": "new", "id": 4, "category_name": "iam"},
  {"code": "V13-07", "title": "Знахідка хмарного аудиту 07", "category_id": 1, "value":
7.0, "status": "done", "id": 7, "category_name": "iam"}
]
```

Фільтрація та пагінація

```
GET /records?min_value=7
HTTP 200
[
  {"code": "V13-01", "title": "Знахідка хмарного аудиту 01", "category_id": 1, "value":
8.0, "status": "new", "id": 1, "category_name": "iam"},
  {"code": "V13-04", "title": "Знахідка хмарного аудиту 04", "category_id": 1, "value":
9.0, "status": "new", "id": 4, "category_name": "iam"},
  {"code": "V13-05", "title": "Знахідка хмарного аудиту 05", "category_id": 2, "value":
7.5, "status": "done", "id": 5, "category_name": "storage"},
  {"code": "V13-07", "title": "Знахідка хмарного аудиту 07", "category_id": 1, "value":
7.0, "status": "done", "id": 7, "category_name": "iam"}
]
```

Фільтрація та пагінація

```
GET /records?category_id=1&status=new
HTTP 200
[
  {"code": "V13-01", "title": "Знахідка хмарного аудиту 01", "category_id": 1, "value":
8.0, "status": "new", "id": 1, "category_name": "iam"},
  {"code": "V13-04", "title": "Знахідка хмарного аудиту 04", "category_id": 1, "value":
9.0, "status": "new", "id": 4, "category_name": "iam"}
]
```

Фільтрація та пагінація

```
GET /records?limit=2&offset=2
HTTP 200
[
  {"code": "V13-03", "title": "Знахідка хмарного аудиту 03", "category_id": 3, "value":
4.0, "status": "done", "id": 3, "category_name": "network"},
  {"code": "V13-04", "title": "Знахідка хмарного аудиту 04", "category_id": 1, "value":
9.0, "status": "new", "id": 4, "category_name": "iam"}
]
```

Фільтрація та пагінація

```
GET /records?q=аудиту%2002
HTTP 200
[
  {"code": "V13-02", "title": "Знахідка хмарного аудиту 02", "category_id": 2, "value":
6.5, "status": "in_progress", "id": 2, "category_name": "storage"}
]
```

Фільтрація та пагінація

```
GET /records?category_id=999
HTTP 200
[]
```

При min_value=7 повернуто V13-01, V13-04, V13-05, V13-07. Одночасні category_id=1 і status=new повертають V13-01 та V13-04. Параметри limit=2, offset=2 повертають записи з id 3 і 4. Відсутня категорія у фільтрі дає порожній список.

Сценарій 5 Невалідні поля і параметри

Невалідні поля

```
POST /records
{"code": "V13-BAD", "title": "Навчальна знахідка", "category_id": 1, "value": -1,
"status": "new"}
HTTP 422
{"detail": [{"type": "greater_than_equal", "loc": ["body", "value"], "msg": "Input should
 be greater than or equal to 0", "input": -1, "ctx": {"ge": 0.0}}]}
```

Невалідні поля

```
POST /records
{"code": "V13-BAD", "title": "   ", "category_id": 1, "value": 7.5, "status": "new"}
HTTP 422
{"detail": [{"type": "string_too_short", "loc": ["body", "title"], "msg": "String should
have at least 3 characters", "input": "   ", "ctx": {"min_length": 3}}]}
```

Невалідні поля

```
POST /records
{"code": "V13-BAD", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "invalid"}
```

```
HTTP 422
{"detail": [{"type": "literal_error", "loc": ["body", "status"], "msg": "Input should be
'new', 'in_progress' or 'done'", "input": "invalid", "ctx": {"expected": "'new',
'in_progress' or 'done'"}}]}
```

Невалідний limit

```
GET /records?limit=0
HTTP 422
{"detail": [{"type": "greater_than_equal", "loc": ["query", "limit"], "msg": "Input
should be greater than or equal to 1", "input": "0", "ctx": {"ge": 1}}]}
```

Після цих запитів стан бази порівняно з попереднім: дані не змінилися. Спеціальний обробник помилок валідації перетворює нескінченне input на рядок, тому відповідь 422 залишається коректним JSON.

Сценарій 6 Верхня межа та нескінченні значення

Перевищення верхньої межі

```
POST /records
{"code": "V13-BAD", "title": "Навчальна знахідка", "category_id": 1, "value": 10.1,
"status": "new"}
HTTP 422
{"detail": [{"type": "less_than_equal", "loc": ["body", "value"], "msg": "Input should be
 less than or equal to 10", "input": 10.1, "ctx": {"le": 10.0}}]}
```

Нескінченне число у JSON

```
POST /records
{"code": "V13-INF", "title": "\u041d\u0430\u0432\u0447\u0430\u043b\u044c\u043d\u0430
\u0437\u043d\u0430\u0445\u0456\u0434\u043a\u0430", "category_id": 1, "value": 1e309,
"status": "new"}
HTTP 422
{"detail": [{"type": "finite_number", "loc": ["body", "value"], "msg": "Input should be a
 finite number", "input": "inf"}]}
```

Після цих запитів стан бази порівняно з попереднім: дані не змінилися. Спеціальний обробник помилок валідації перетворює нескінченне input на рядок, тому відповідь 422 залишається коректним JSON.

Сценарій 7 Неіснуючі ресурси

Відсутній запис

```
GET /records/999
HTTP 404
{"detail": "Запис не знайдено"}
```

Відсутня категорія

```
POST /records
{"code": "V13-NOCAT", "title": "Навчальна знахідка", "category_id": 999, "value": 7.5,
"status": "new"}
HTTP 404
{"detail": "Категорію не знайдено"}
```

Сценарій 8 Конфлікти унікальності

Повторний code

```
POST /records
{"code": "V13-01", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new"}
HTTP 409
{"detail": "Конфлікт даних: дублікат або категорія містить записи"}
```

Повторна назва категорії

```
POST /categories
{"name": "iam"}
HTTP 409
{"detail": "Конфлікт даних: дублікат або категорія містить записи"}
```

Сценарій 9 Видалення категорій

Захист непорожньої категорії

```
DELETE /categories/1
HTTP 409
{"detail": "Конфлікт даних: дублікат або категорія містить записи"}
```

Створення порожньої категорії

```
POST /categories
{"name": "temporary"}
HTTP 201
{"name": "temporary", "id": 4}
```

Видалення порожньої категорії

```
DELETE /categories/4
HTTP 204
(порожнє тіло відповіді)
```

Сценарій 10 Коректний пакет

Коректний пакет

```
POST /records/bulk
[{"code": "V13-BULK1", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new"}, {"code": "V13-BULK2", "title": "Навчальна знахідка", "category_id": 2,
"value": 5, "status": "new"}]
HTTP 201
[
  {"code": "V13-BULK1", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new", "id": 9, "category_name": "iam"},
  {"code": "V13-BULK2", "title": "Навчальна знахідка", "category_id": 2, "value": 5.0,
"status": "new", "id": 10, "category_name": "storage"}
]
```

Сценарій 11 Пакет із дублікатом коду

Скасування всього пакета

```
POST /records/bulk
[{"code": "V13-ROLLBACK11", "title": "Навчальна знахідка", "category_id": 1, "value":
7.5, "status": "new"}, {"code": "V13-01", "title": "Навчальна знахідка", "category_id":
1, "value": 7.5, "status": "new"}]
HTTP 409
{"detail": "Конфлікт даних: дублікат або категорія містить записи"}
```

```
{
  "count_before": 10,
```

```
  "count_after": 10,
  "first_code": "V13-ROLLBACK11",
  "first_code_exists": false
}
```

Сценарій 12 Пакет із відсутньою категорією

Скасування всього пакета

```
POST /records/bulk
[{"code": "V13-ROLLBACK12", "title": "Навчальна знахідка", "category_id": 1, "value":
7.5, "status": "new"}, {"code": "V13-NOCAT", "title": "Навчальна знахідка",
"category_id": 999, "value": 7.5, "status": "new"}]
HTTP 404
{"detail": "Категорію не знайдено"}
```

```
{
  "count_before": 10,
  "count_after": 10,
  "first_code": "V13-ROLLBACK12",
  "first_code_exists": false
}
```

Сценарій 13 Спроба SQL ін’єкції

SQL-ін'єкція як буквальний підрядок

```
GET /records?q=%27%20OR%201%3D1%20--
HTTP 200
[]
```

Пошуковий рядок передано окремим SQL-параметром. instr(title, ?) виконує буквальний пошук, тому введений фрагмент не змінює структуру запиту. Записи не змінено; API продовжує роботу.

Сценарій 14 Збереження після перезапуску

Збереження перед перезапуском

```
POST /records
{"code": "V13-PERSIST", "title": "Навчальна знахідка", "category_id": 3, "value": 7.5,
"status": "new"}
HTTP 201
{"code": "V13-PERSIST", "title": "Навчальна знахідка", "category_id": 3, "value": 7.5,
"status": "new", "id": 11, "category_name": "network"}
```

Читання після перезапуску процесу

```
GET /records/11
HTTP 200
{"code": "V13-PERSIST", "title": "Навчальна знахідка", "category_id": 3, "value": 7.5,
"status": "new", "id": 11, "category_name": "network"}
```

Підсумкові дані

```
GET /records?limit=100
HTTP 200
[
  {"code": "V13-01", "title": "Знахідка хмарного аудиту 01", "category_id": 1, "value":
8.0, "status": "new", "id": 1, "category_name": "iam"},
  {"code": "V13-02", "title": "Знахідка хмарного аудиту 02", "category_id": 2, "value":
6.5, "status": "in_progress", "id": 2, "category_name": "storage"},
  {"code": "V13-03", "title": "Знахідка хмарного аудиту 03", "category_id": 3, "value":
```

```
4.0, "status": "done", "id": 3, "category_name": "network"},
  {"code": "V13-04", "title": "Знахідка хмарного аудиту 04", "category_id": 1, "value":
9.0, "status": "new", "id": 4, "category_name": "iam"},
  {"code": "V13-05", "title": "Знахідка хмарного аудиту 05", "category_id": 2, "value":
7.5, "status": "done", "id": 5, "category_name": "storage"},
  {"code": "V13-06", "title": "Знахідка хмарного аудиту 06", "category_id": 3, "value":
3.0, "status": "in_progress", "id": 6, "category_name": "network"},
  {"code": "V13-07", "title": "Знахідка хмарного аудиту 07", "category_id": 1, "value":
7.0, "status": "done", "id": 7, "category_name": "iam"},
  {"code": "V13-08", "title": "Знахідка хмарного аудиту 08", "category_id": 2, "value":
5.0, "status": "new", "id": 8, "category_name": "storage"},
  {"code": "V13-BULK1", "title": "Навчальна знахідка", "category_id": 1, "value": 7.5,
"status": "new", "id": 9, "category_name": "iam"},
  {"code": "V13-BULK2", "title": "Навчальна знахідка", "category_id": 2, "value": 5.0,
"status": "new", "id": 10, "category_name": "storage"},
  {"code": "V13-PERSIST", "title": "Навчальна знахідка", "category_id": 3, "value": 7.5,
"status": "new", "id": 11, "category_name": "network"}
]
```

Перший процес uvicorn завершено, після чого запущено новий процес з тим самим SQLite-файлом. Відповідь GET після перезапуску повністю збігається зі створеним записом V13-PERSIST.

8 Демонстрація скасування пакета

create_records відкриває одну транзакцію BEGIN для всього пакета. Після вставки першого елемента помилка другого виходить із with connection, що викликає rollback. Окремих commit усередині циклу немає. З’єднання закривається contextlib.closing після завершення транзакції.

Перевірка	До	Після	Перший елемент
Дублікат V13-01, HTTP 409	10	10	V13-ROLLBACK11 відсутній
category_id=999, HTTP 404	10	10	V13-ROLLBACK12 відсутній

Сценарії порівнюють повні знімки записів до і після запиту та окремо перевіряють відсутність code першого елемента. Додаткові автоматизовані перевірки охоплюють дублікат усередині пакета, порожній пакет і пакет із 101 елемента, невідомі поля, неповний PUT, буквальні % та _, відмінність регістру, граничні value=0 і value=10, прямі обмеження SQLite та некоректний seed.

9 Перевірка коду за допомогою Ruff

Застосовано правила з наявного ruff.toml: E, W, F, I, N, line-length=79. Налаштування збережено у ruff.toml. Наведено фактичні результати фінальної перевірки; окремо виконано автоматизовані тести поведінки API та транзакцій.

```
$ ruff check labs/lab03
All checks passed!

$ ruff format --check labs/lab03
11 files already formatted

$ ruff check .
All checks passed!

$ ruff format --check .
24 files already formatted

$ python -m unittest discover -v
```

```
Ran 11 tests
OK

$ python -m tests.verify_http
PASS: 14 сценаріїв, 37 кроків. Перезапуск процесу перевірено.
```

Повний журнал unittest збережено в output/evidence/unittest.txt. Повідомлення про порушення UNIQUE та FOREIGN KEY у журналі очікувані: вони виникають у негативних тестах, для яких клієнт отримує 409.

10 Скріншоти

Знімки зроблено під час роботи локального API з початковими даними варіанта 13.

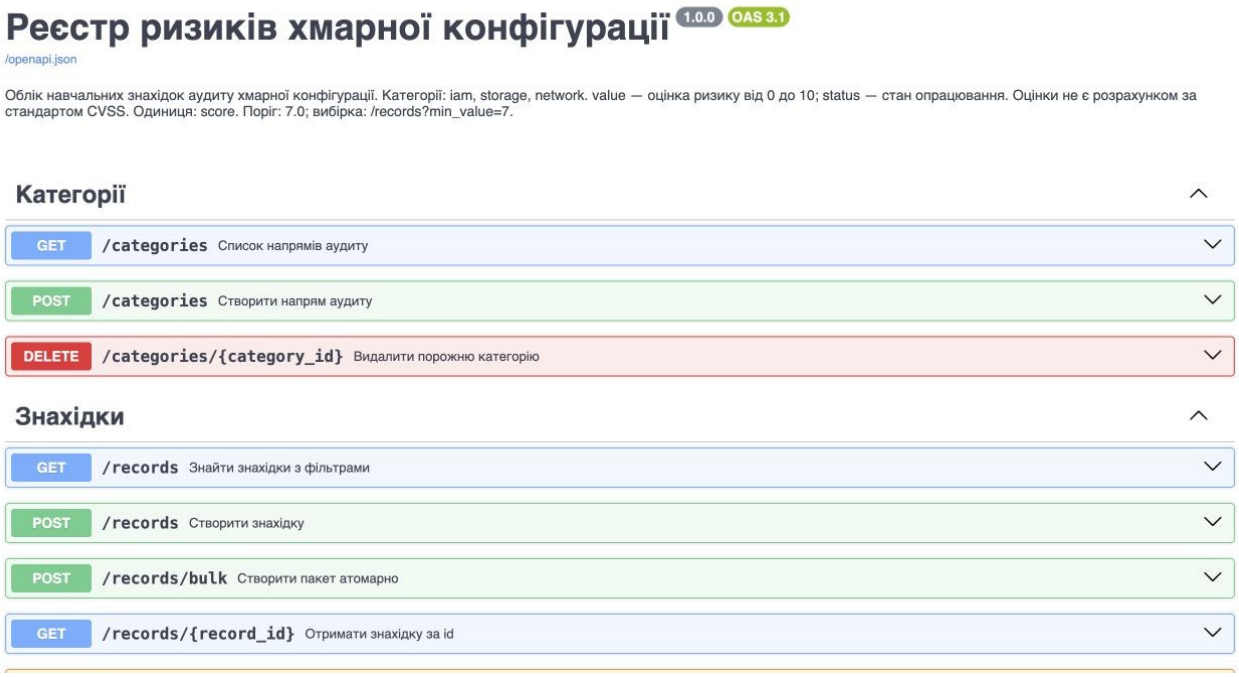


Рисунок 1 — Swagger UI реєстру ризиків хмарної конфігурації

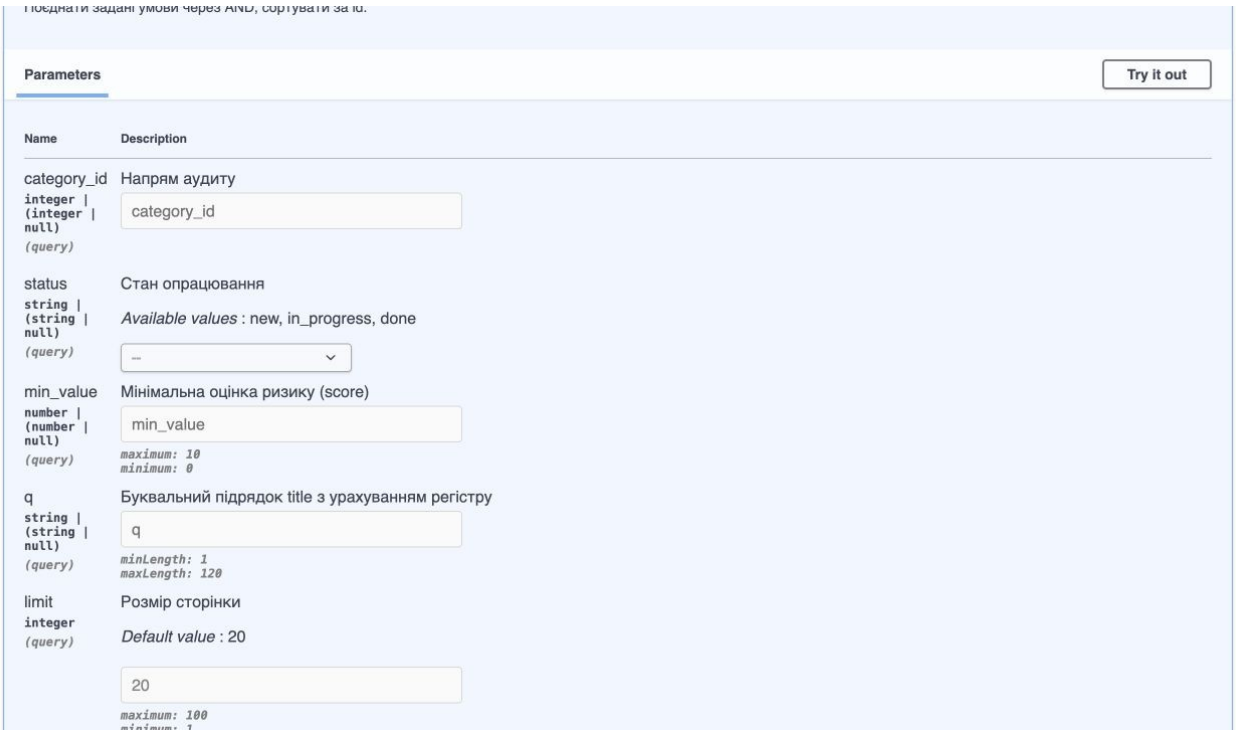


Рисунок 2 — Фільтри та обмеження параметрів GET /records

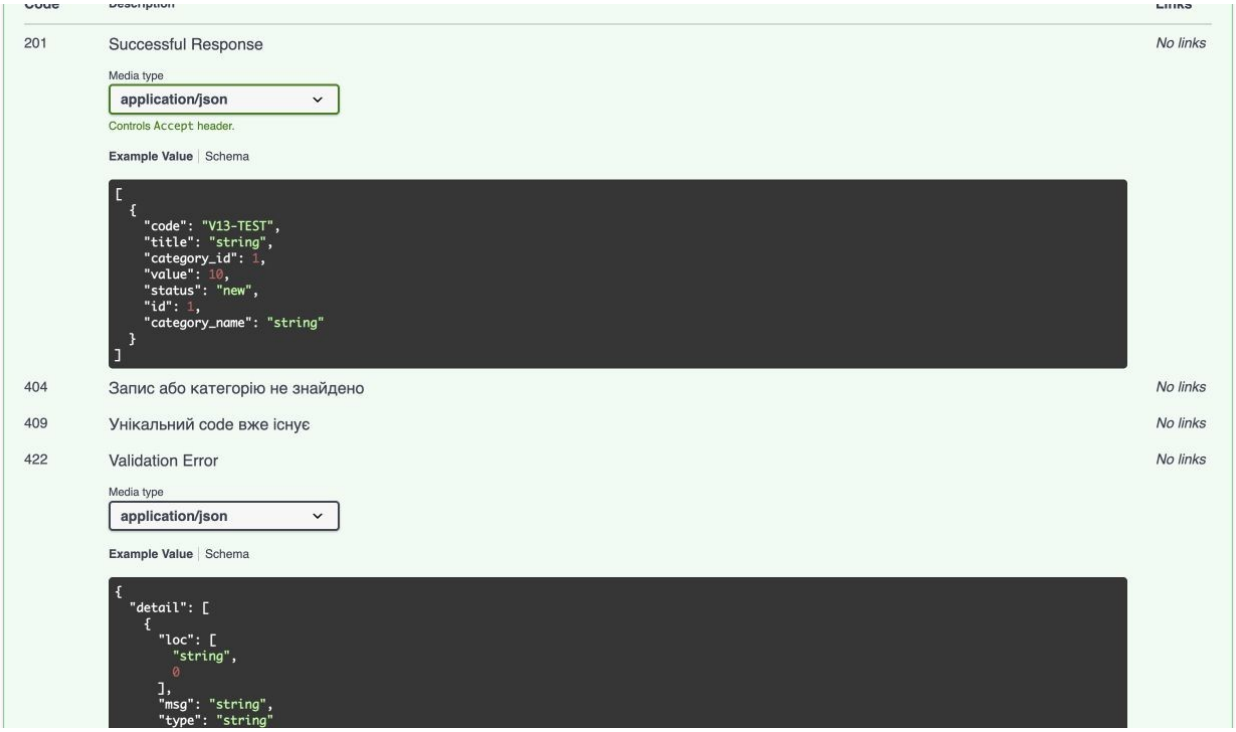


Рисунок 3 — Відповіді пакетної операції 201, 404, 409 та 422

11 Відповіді на контрольні питання

1. Таблиця, запис, поле та зв’язок. Таблиця містить однотипні об’єкти, запис описує один об’єкт, поле — його властивість. categories описує напрями аудиту, records — знахідки. Одній категорії відповідає багато знахідок.

2. Первинний та зовнішній ключі. Первинний id однозначно визначає рядок. Зовнішній records.category_id посилається на categories.id. Неіснуюча категорія

при записі спричиняє 404, а FOREIGN KEY додатково забороняє неузгоджене посилання.

3. SQLite порівняно з CSV та JSON. SQLite підтримує SQL-вибірки, JOIN, індекси, обмеження та транзакції у файловій базі. CSV і JSON є форматами даних; пошук, зв’язки й узгоджені оновлення в них потребують окремої реалізації.

4. Параметризовані запити. Команда SQL і дані передаються окремо. Фрагмент OR 1=1 залишається значенням параметра. Знак ? підставляє значення, а не назву таблиці, стовпця чи оператор; структуру запиту формують лише довірені фіксовані фрагменти.

5. Транзакція для пакета. Одна транзакція зберігає всі елементи або скасовує всі зміни. commit після кожного елемента залишив би перший запис у базі, навіть якщо другий порушив обмеження.

6. Контекстний менеджер з’єднання. with connection підтверджує відкриту транзакцію за успіху та скасовує її за винятку. Сам вхід у цей блок не починає транзакцію і не закриває з’єднання. Тут BEGIN задається явно для пакета, а closing забезпечує close.

7. HTTP та SQL. GET відповідає SELECT, POST — INSERT, PUT — UPDATE, DELETE — DELETE. У цій роботі PUT повністю замінює всі п’ять змінюваних полів, тому кожне з них обов’язкове; id зберігається.

8. Шлях, параметри та тіло. /records/9 містить id у шляху. /records?category_id=1&status=new передає умови вибірки. JSON-тіло POST або PUT задає дані створення чи заміни.

9. Перевірки Pydantic. Моделі перевіряють типи, довжини після обрізання пробілів, додатні id, скінченне value у межах 0–10, допустимий status, відсутність зайвих полів і розмір пакета 1–100.

10. Призначення response_model. Модель відповіді задає структуру результату, перевіряє й фільтрує дані, що повертаються, та формує їх опис в OpenAPI.

11. Хто створює id. Ідентифікатор створює SQLite. Клієнт не задає його у POST чи PUT; extra="forbid" відхиляє таке поле з кодом 422.

12. Приклади HTTP кодів. 201 — створення знахідки; 204 — її видалення; 404 — відсутній id; 409 — повторний code або непорожня категорія; 422 — value=-1. Статус 204 означає успішне виконання без тіла відповіді.

13. Файли в Git. Код, SQL-схема та seed.json дозволяють відтворити базу. Робочий SQLite-файл є змінним бінарним результатом і може містити поточні дані, тому його виключено з історії Git.

12 Висновки

Під час лабораторної роботи реалізовано реєстр ризиків хмарної конфігурації для варіанта 13 на FastAPI та SQLite без ORM. Створено API для категорій і знахідок, повний цикл CRUD, JOIN із назвою категорії, фільтрацію, буквальний пошук, пагінацію та пакетне додавання.

Вхідні дані перевіряє Pydantic, а узгодженість у SQLite підтримують CHECK, UNIQUE, FOREIGN KEY і транзакції. Усі введені значення передаються через SQL-параметри. Непорожні категорії захищено від видалення, а повторний seed не змінює дані.

Виконано 11 автоматизованих тестів і всі 14 сценаріїв методички через HTTP. Підтверджено повне скасування пакетів при помилці другого елемента та збереження запису після перезапуску процесу сервера. Ruff check і Ruff format --check завершилися успішно.